

Implementation Document

1. The General Structure of the Program

- read_articles: returns a string made of all of the articles combined
- clean_articles: carries out preprocessing (converts to lowercase, removes html tags, urls, whitespace and punctuations) and returns a list of tokens
- class Node: used to create the trie tree-structure and possesses attributes 'children' and 'count'.
- class TrieTree: stores a sequence of words in a trie tree structure that can be used to determine successor words given a sequence
- Markov_model: builds an n-gram Markov model that is stored in a trie
- text_generation: generates sentences of a given length and number using the Markov model

2. Achieved Time Complexity

Function	Time Complexity
read_articles	O(n)
clean_articles	O(n)
TrieTree.add_sequence()	O(n)
TrieTree.get_successors()	O(n)
markov_model	
text_generation	

3. Possible shortcomings and suggestions for improvements

- More work is required to improve the sentence intelligibility of the generated text (i.e part-of-speech tagging)
- The parser would frequently fail to identify field separators so functions that can handle delimiters are needed.
- Preprocessing in clean_articles needs to be finetuned so that no isolated characters are found in the generated text.

4. Use of large language models

Claude was used for the following:

- Debugging implementation issues
- Clarifying Markov chain fundamentals
- Understanding trie data structures

5. List of the sources you have used, only those relevant to your work.

<https://www.geeksforgeeks.org/nlp/markov-chains-in-nlp/>

https://en.wikipedia.org/wiki/Markov_chain

<https://www.geeksforgeeks.org/dsa/trie-insert-and-search/>

<https://en.wikipedia.org/wiki/Trie>